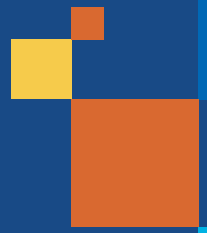
Netdev 0x15

Dynamic Network Interface Power Management aka PowerMAN

Jesse Brandeburg, Brett Creeley

Intel Ethernet Products Group (EPG)



intel®

Legal Disclaimers

- Performance varies by use, configuration and other factors. Learn more at https://www.intel.com/PerformanceIndex

- Performance results are based on testing as of dates shown in configurations and may not reflect all publicly available updates. See backup for configuration details. No product or component can be absolutely secure.

- Your costs and results may vary.

- Intel technologies may require enabled hardware, software or service activation.

- © Intel Corporation. Intel, the Intel logo, and other Intel marks are trademarks of Intel Corporation or its subsidiaries. Other names and brands may be claimed as the property of others.

Who are we?

Intel Ethernet Products Group

- Jesse Brandeburg
  - Principal Software Engineer
- Brett Creeley
  - Software Engineer



Agenda



Photo by Crawford Jolly on Unsplash

- Background
- Problem description
- Proof of concept (powerMAN!)

Power Management

- Today's normal configuration

Workload	Strategy	Result
Latency Sensitive	Poll or turn off power management	Idle Xeon 8180 (C0) ~275 watts CPU and 45 watts memory
Throughput	Enable "enough" queues to keep up, leave turbo enabled	Latency is bad, packets might be dropped (Idle ~28 watts CPU, 5 watts mem)





Background on Packet Drops



- Dropped packets in Ethernet packet delivery

1. Dropped from the controller FIFO
2. Dropped from the driver's ring due to no available slots
3. Dropped from the stack due to buffers full or other reasons

- Cause:

1. Controller DMA too slow or blocked
2. CPU too slow to catch up, or already busy, or too much work per packet
3. Myriad – usually too small socket buffers to cover scheduling delays

C-states vs PC-states (Xeon focused)



- PC-state
  - “Package C-state”, means the uncore with the DMA controller goes to sleep
  - Prevents DMA to the host
  - The only network buffering is the FIFO (integrated memory) on the ethernet controller
  - Wake from PC6 to PC2 can take 40us (typical time)
  - Ethernet stat tied to dropped packets from this cause is “port drops”
    - Port Drops come from anything that makes DMA take too long, not just power events
  - # turbostat -S

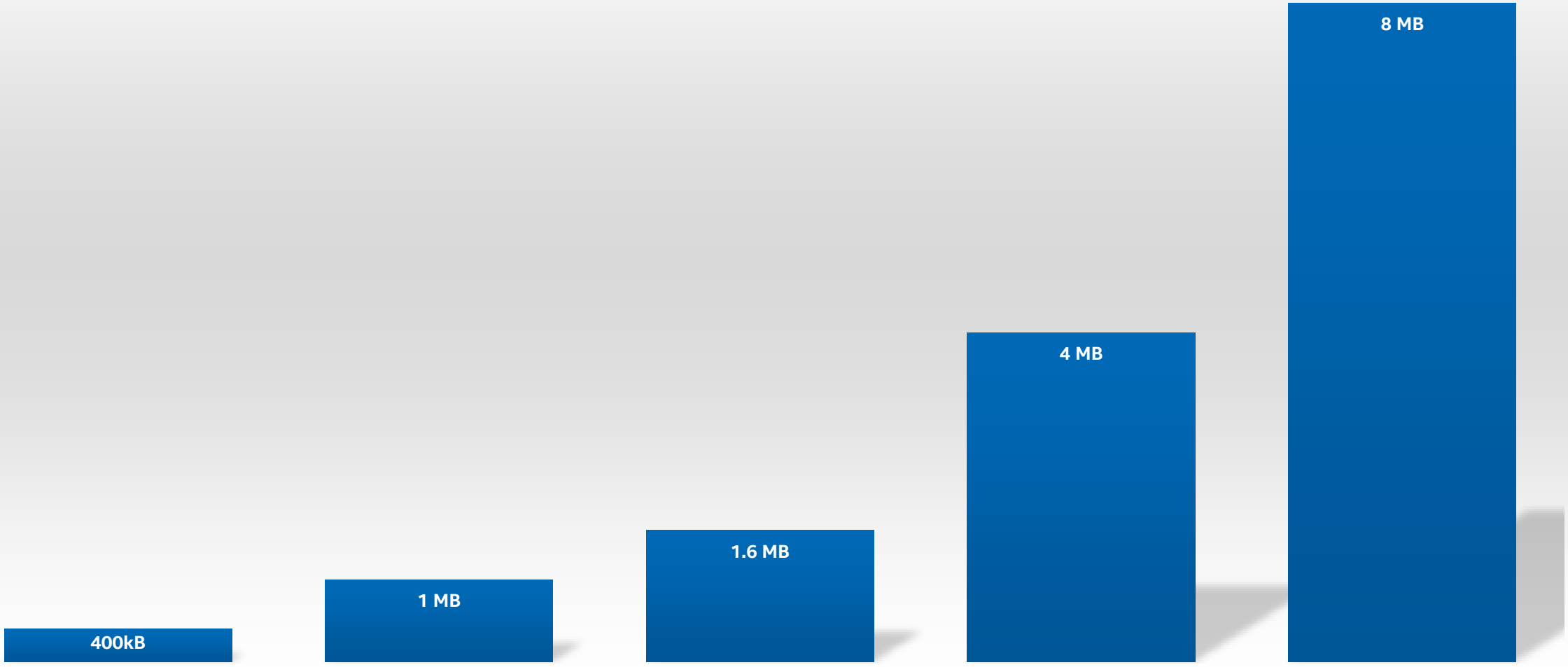
C-states vs PC-states (cont.)



- C-states
  - Typically referred to as C0 through C6 - servers
    - Client machines show as high as C10, with exit latencies at 890us!
  - Core is halted when sleeping, with varying degrees of power savings that result in longer exit times for more savings
  - Interrupts are enabled (but handling is delayed by wake time)
  - DMA can flow
  - Ethernet stat tied to dropped packets from this cause is “VSI drops”
    - VSI drops come whenever a queue has no more resources for receive
  - # cpupower idle-info

On-chip storage is limited, and growing slower than data rates

Per port **mega-bytes** of memory needed to buffer 40us wake delay during wire speed burst of receive packets - 8 traffic classes



Using Megabytes of memory doesn't solve the problem

Networking is the only high speed I/O with unpredictable arrival events

Rx Storage in microseconds with 40us wake delay

100Gb Ethernet, Single port active, Single Queue

Ring Size	Packet Size							# of receive Q
	64	128	256	512	1024	1518	9000	112
	0.00672	0.01184	0.02208	0.04256	0.08352	0.12304	0.741761	
64	0.43	0.76	1.41	2.72	5.35	7.87	47.47	14,909,440
128	0.86	1.52	2.83	5.45	10.69	15.75	94.95	29,818,880
256	1.72	3.03	5.65	10.90	21.38	31.50	189.89	59,637,760
512	3.44	6.06	11.30	21.79	42.76	63.00	379.78	119,275,520
1024	6.88	12.12	22.61	43.58	85.52	125.99	759.56	238,551,040
2048	**13.76**	**24.25**	**45.22**	**87.16**	**171.05**	**251.99**	**1519.13**	**477,102,080**
4096	27.53	48.50	90.44	174.33	342.10	503.97	3038.25	954,204,160
8160	54.84	96.61	180.17	347.29	681.52	1004.01	6052.77	1,900,953,600

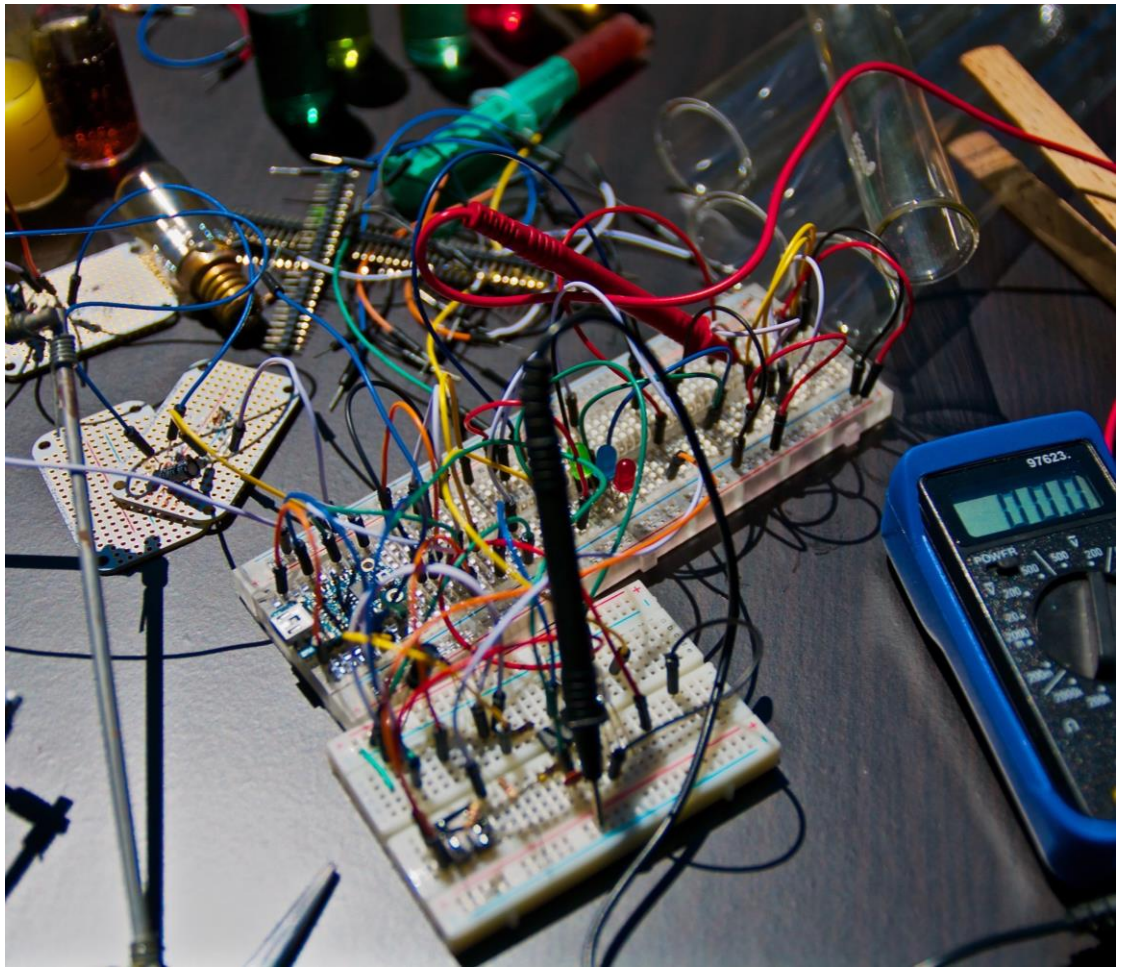
Rx Storage in microseconds with 40us wake delay

100Gb Ethernet, Single port active, Single Queue

Ring Size	Packet Size							# of receive Q
	64	128	256	512	1024	1518	9000	112
	0.00672	0.01184	0.02208	0.04256	0.08352	0.12304	0.741761	
64	0.43	0.76	1.41	2.72	5.35	7.87	47.47	14,909,440
128	0.86	1.52	2.83	5.45	10.69	15.75	94.95	29,818,880
256	1.72	3.03	5.65	10.90	21.38	31.50	189.89	59,637,760
512	3.44	6.06	11.30	21.79	42.76	63.00	379.78	119,275,520
1024	6.88	12.12	22.61	43.58	85.52	125.99	759.56	238,551,040
2048	**13.76**	**24.25**	**45.22**	**87.16**	**171.05**	**251.99**	**1519.13**	**477,102,080**
4096	27.53	48.50	90.44	174.33	342.10	503.97	3038.25	954,204,160
8160	54.84	96.61	180.17	347.29	681.52	1004.01	6052.77	1,900,953,600

As network speeds increase, kernel needs to adapt

What should we do?



- Let's design an experiment to see if we can save power, and avoid suffering from extra latency or dropped packets
- Proof of concept
  - Bash scripts to run the setup and logic
  - Consolidate RSS table to one CPU at idle and keep that CPU at C1
    - Reduce latency, be efficient if possible
  - As load increases, use RSS to direct to more queues

Test setup

- Back-to-back with 100Gb Intel Ethernet Network Adapter E810-CQDA2, Mellanox CX4
- Intel Driver (latest SourceForge release)
- Mellanox Driver – From Kernel
- Netperf 2.7.0

	Configuration
SUT Setup	
Platform	S2600STQ
# Nodes	1
# Sockets	2
CPU	Intel® Xeon® Platinum 8180 Processor 2.50GHz
Cores/socket, Threads/socket	28/56
Microcode	0x2006a0a
HT	On
Turbo	On
Power management	Enabled
BIOS version	SE5C620.86B.00.01.0015.110720180833
System DDR Mem Config: slots / cap / speed	12 slots / 8GB / 2666
Total Memory/Node (DDR, DCPMM)	96GB
PCH	Intel C620
OS	Fedora 32
Version	5.11.22-100.fc32

Proof of Concept

- 2 bash scripts (TBD published on github)
- First sets up the network with “stable settings”
  - Goal is to avoid test to test variance due to remote node or scheduler
- Second runs the PowerMAN logic

The Experiment Setup

- Baseline
  - System in C1 (~60 Watts)
  - Local node queues only
  - RSS set to equal weight across all queues
  - Interrupt rate set to a constant 10us
- PowerMAN
  - System in C6 (all sleep states enabled)
  - Local node queues only
  - RSS directing to 1 queue / core
  - Script managing power state, putting RSS cores in C1
  - Interrupt rate set to a constant 10us

PowerMAN Script Flow

Initialization

find number of enabled queues on eth0

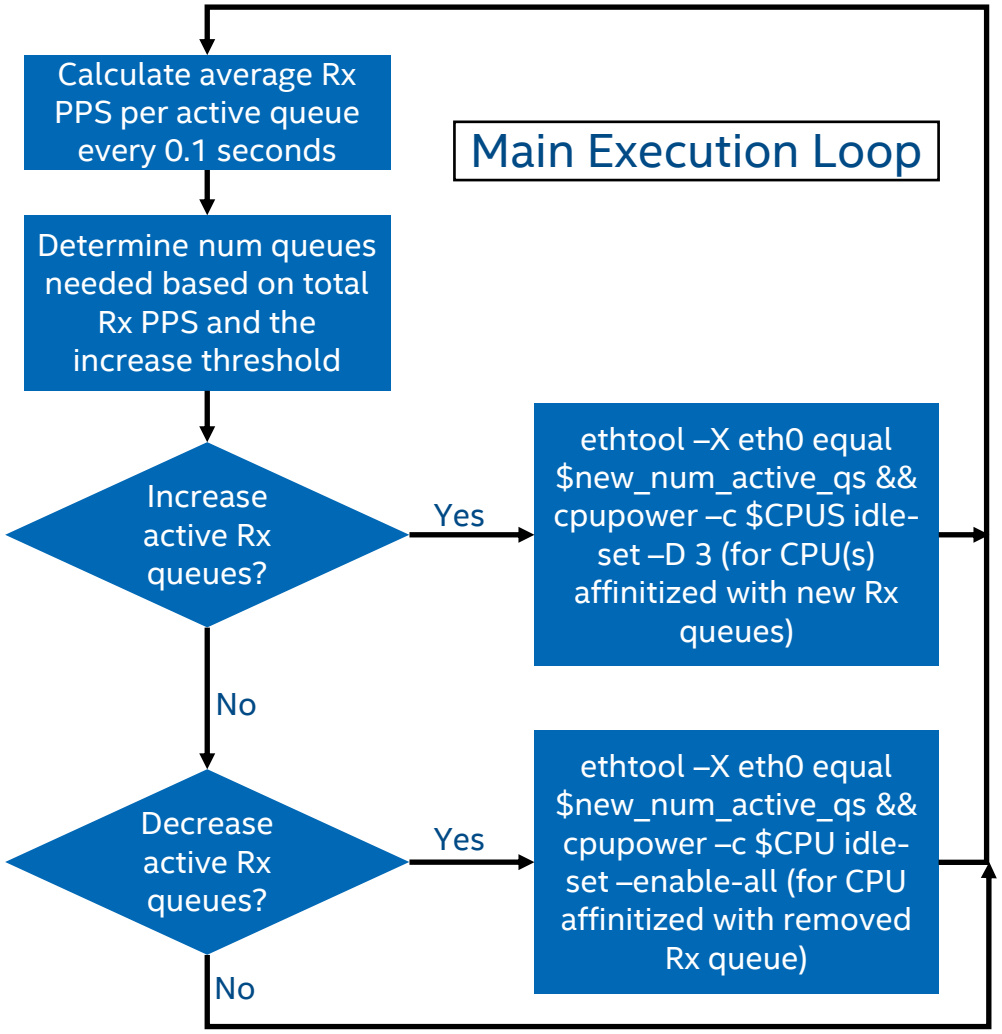
ethtool –X eth0 equal 1

find IRQ affinitized CPUs

Enable all C-States for all IRQ affinitized CPUs except the active CPU – Disable all but C0/C1 for active queue

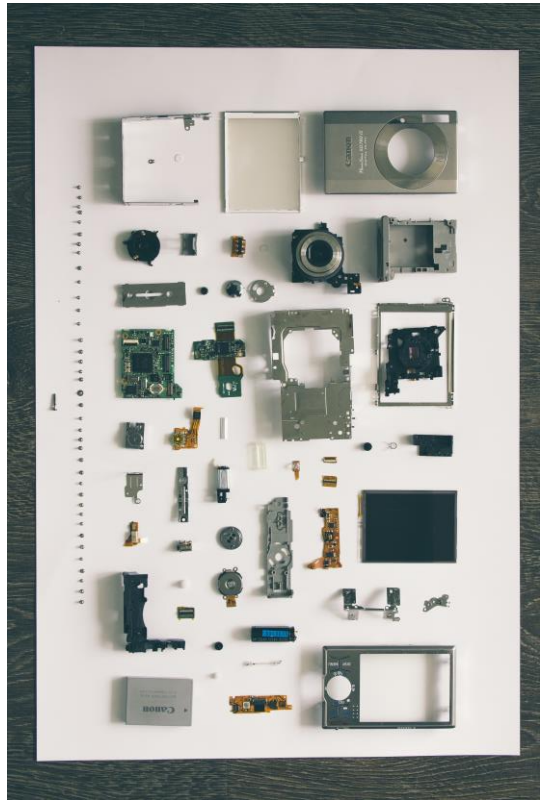
Set increase/decrease PPS threshold

Main Execution Loop



The Tests

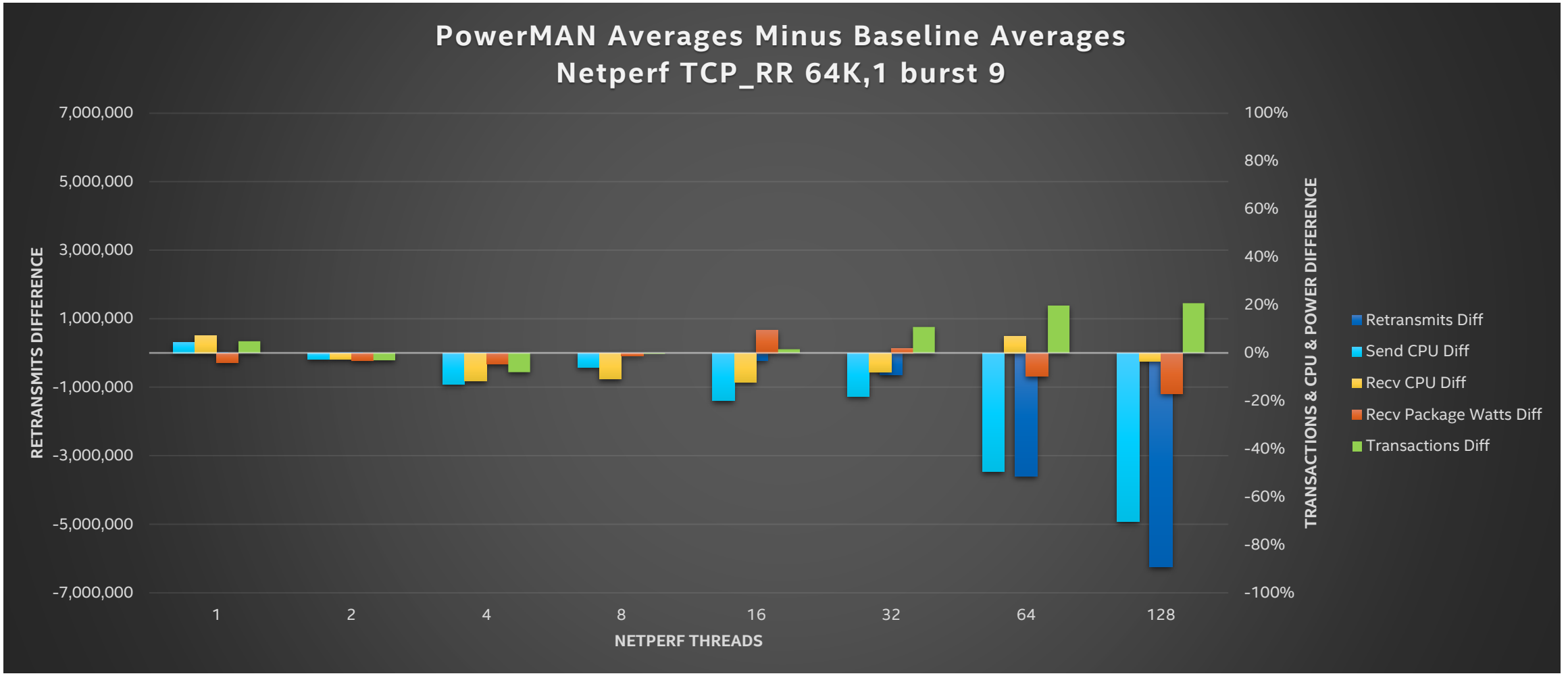
- Netperf TCP_RR (burst 9)
  - Send – SUT (system under test) is sending 64KB buffers, 1 byte receives
  - Recv – SUT is receiving 64KB buffers, 1 byte sends
  - Bi-directional – SUT is sending and receiving 64KB buffers
- Netperf Stream
  - Send – SUT is receiving 16KB (default) buffers, sending ACKs
- Threads
  - Iterated 30 second runs of 1,2,4,8,16,32,64,128 concurrent threads
- Iterations
  - 11 runs for each test, averaged results



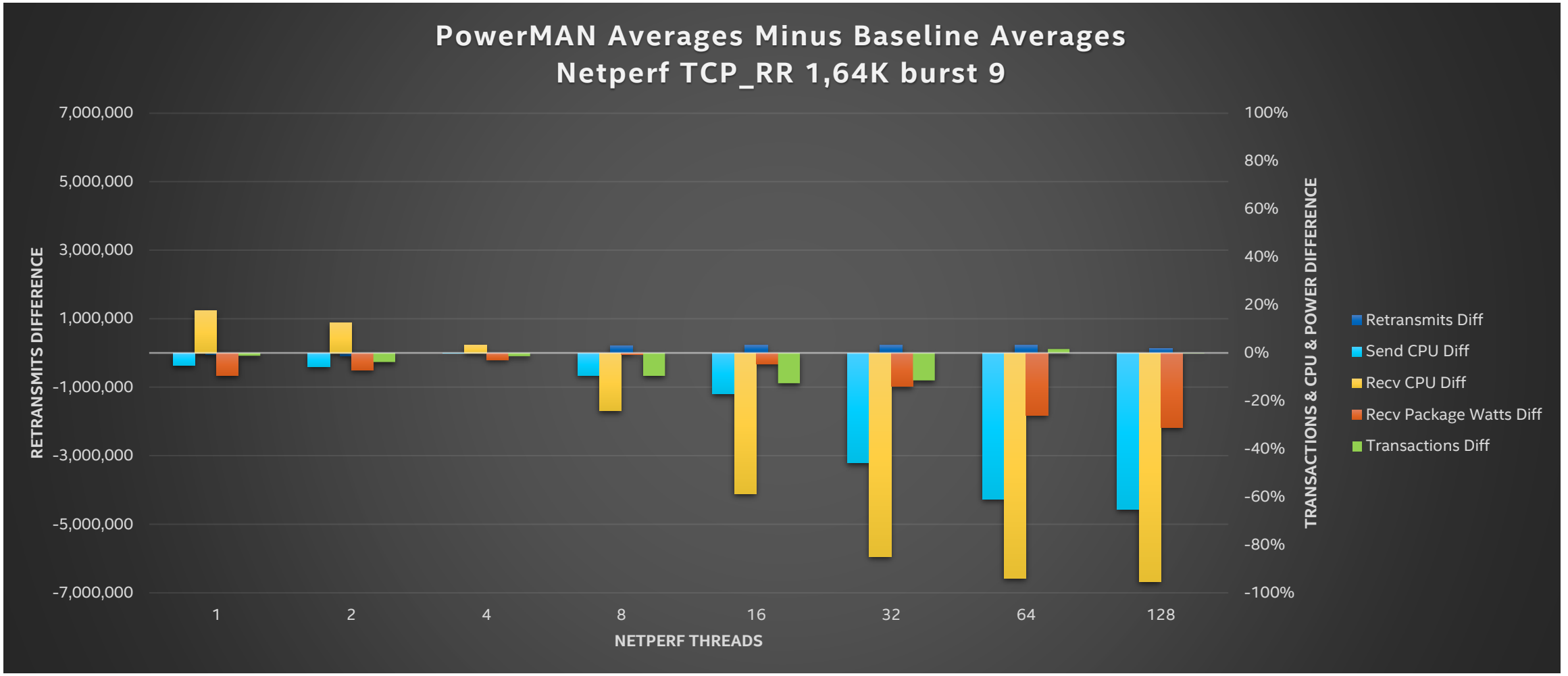
Reading the Charts & Understanding the Data

- The following charts take the differences between PowerMAN and baseline runs to highlight the positives/negatives of the experiment
- Retransmits Diff
  - PowerMAN Retransmits – Baseline Retransmits
  - <0 is a positive result for PowerMAN and >0 is a negative result for PowerMAN
- Send/Recv CPU & Recv Watts Diff
  - (PowerMAN – Baseline) / ((PowerMAN + Baseline) / 2)
  - <0 is a positive result for PowerMAN and >0 is a negative result for PowerMAN
- Transactions & Throughput Diff
  - (PowerMAN – Baseline) / ((PowerMAN + Baseline) / 2)
  - >0 is a positive result for PowerMAN and <0 is a negative result for PowerMAN

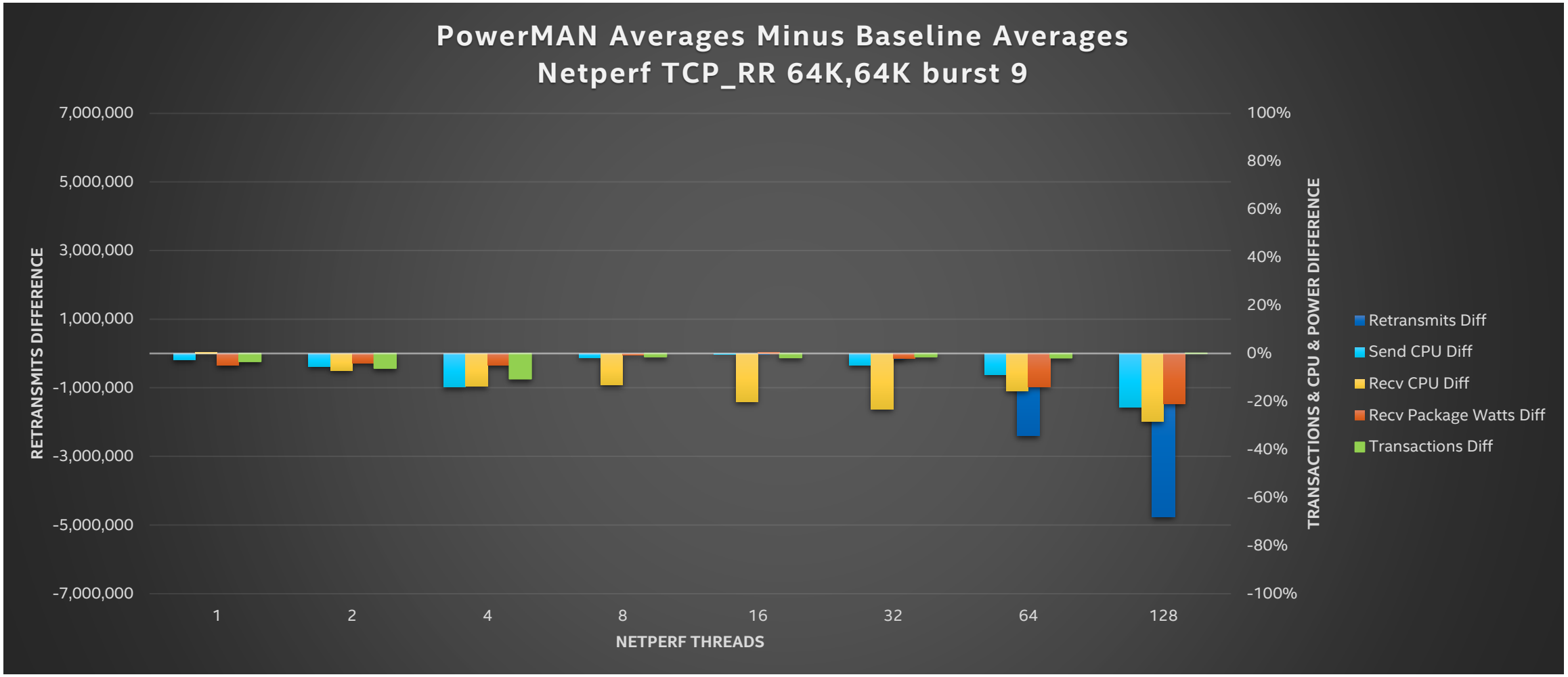
PowerMAN Minus Baseline - Receive



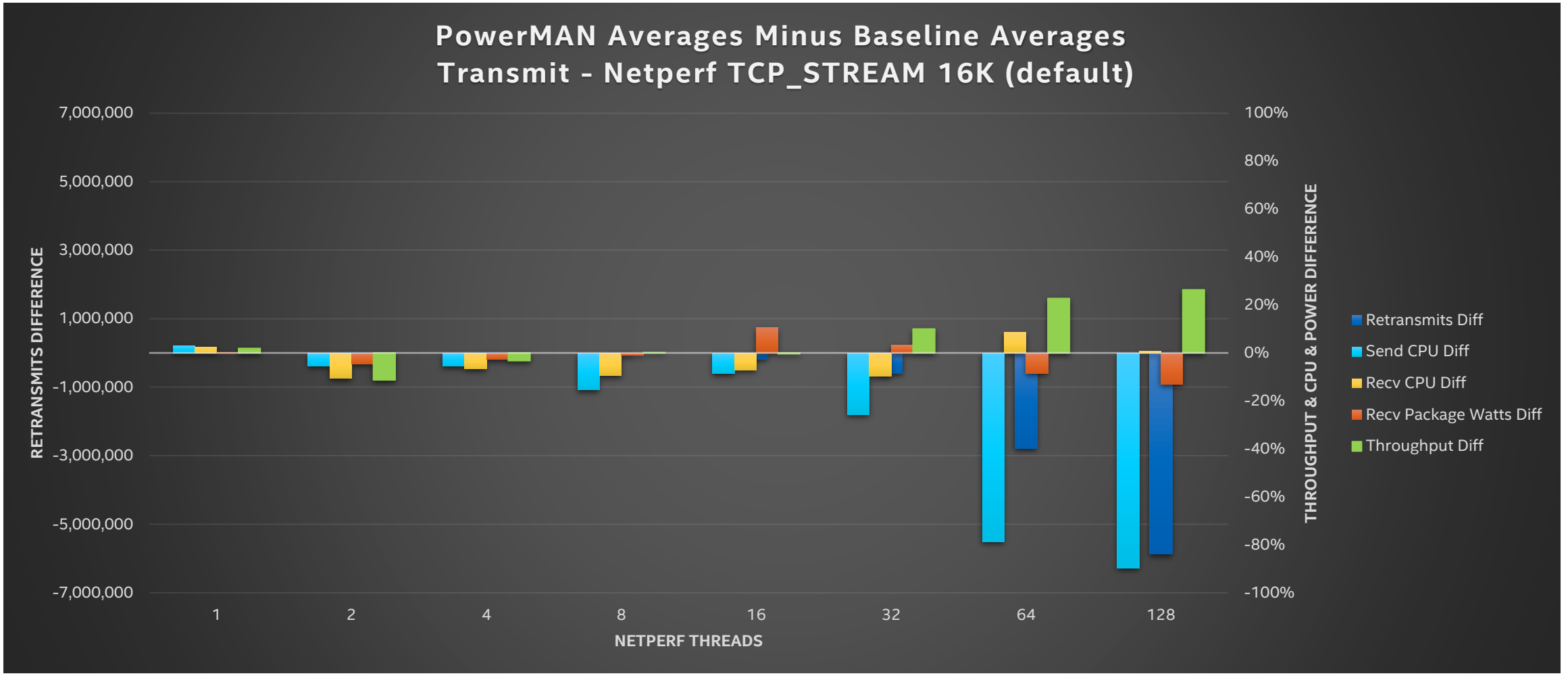
PowerMAN Minus Baseline - Transmit



PowerMAN Minus Baseline – Bi-directional



PowerMAN Minus Baseline – Stream Receive

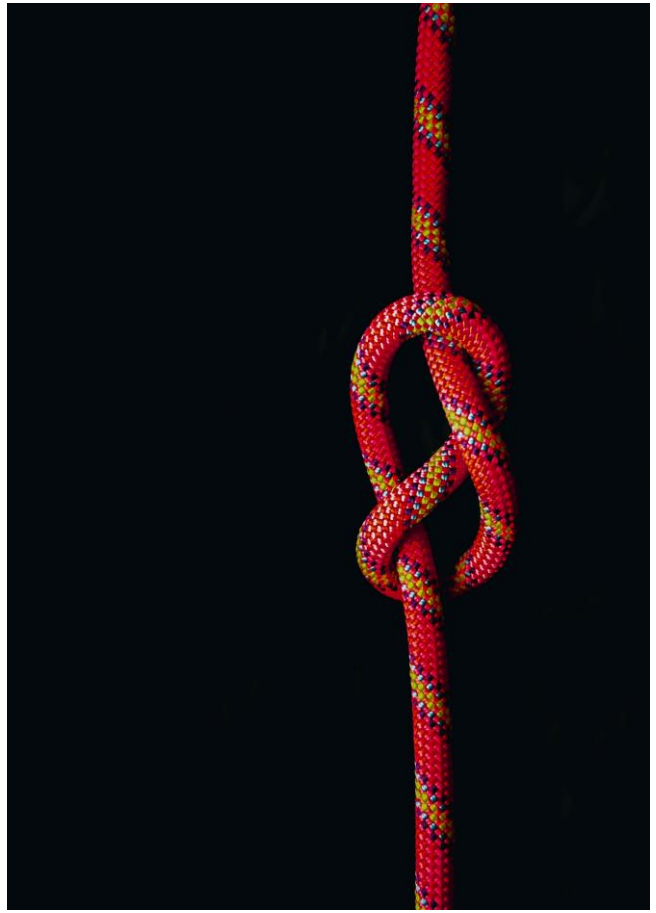


Experiment Conclusions

- PowerMAN prevents retransmits due to drops
- Best response from PowerMAN is in high thread count tests
- Marginal power was saved in lower thread count tests
- Efficiency is increased by workload consolidation to fewer queues
- Transmit focused workloads had reduced performance
  - PoC was 100% receive focused (measure/react)
  - Transmit CPUs were going to sleep frequently, only 1 receive queue active

General Conclusions

- Reducing retransmits / drops reduces power use
- If compared to C0 (poll) power, power savings would be larger
  - Future - Run the baseline in C0 (poll)
- Future - focus on both transmit and receive
- Future - expand to cover more tests
  - XDP, AF_XDP, packet generator, forwarding workloads, UDP, busy poll



Next Steps



- Userspace or Kernel?
  - Python script
  - Scheduler changes
- Scheduler gains networking inputs
  - Becomes aware of cycles spent in a NAPI handler (threaded NAPI?)
  - Knows what queues are part of which netdev
  - Knows about queue overruns (drops) or full queues
  - Knows about DIMlib setting changes?
- New Scheduling algorithm
  - Integrates data of cycles used, and packet drops, to choose when to enable more RSS queues to balance the workload

Takeaways

Final Thoughts

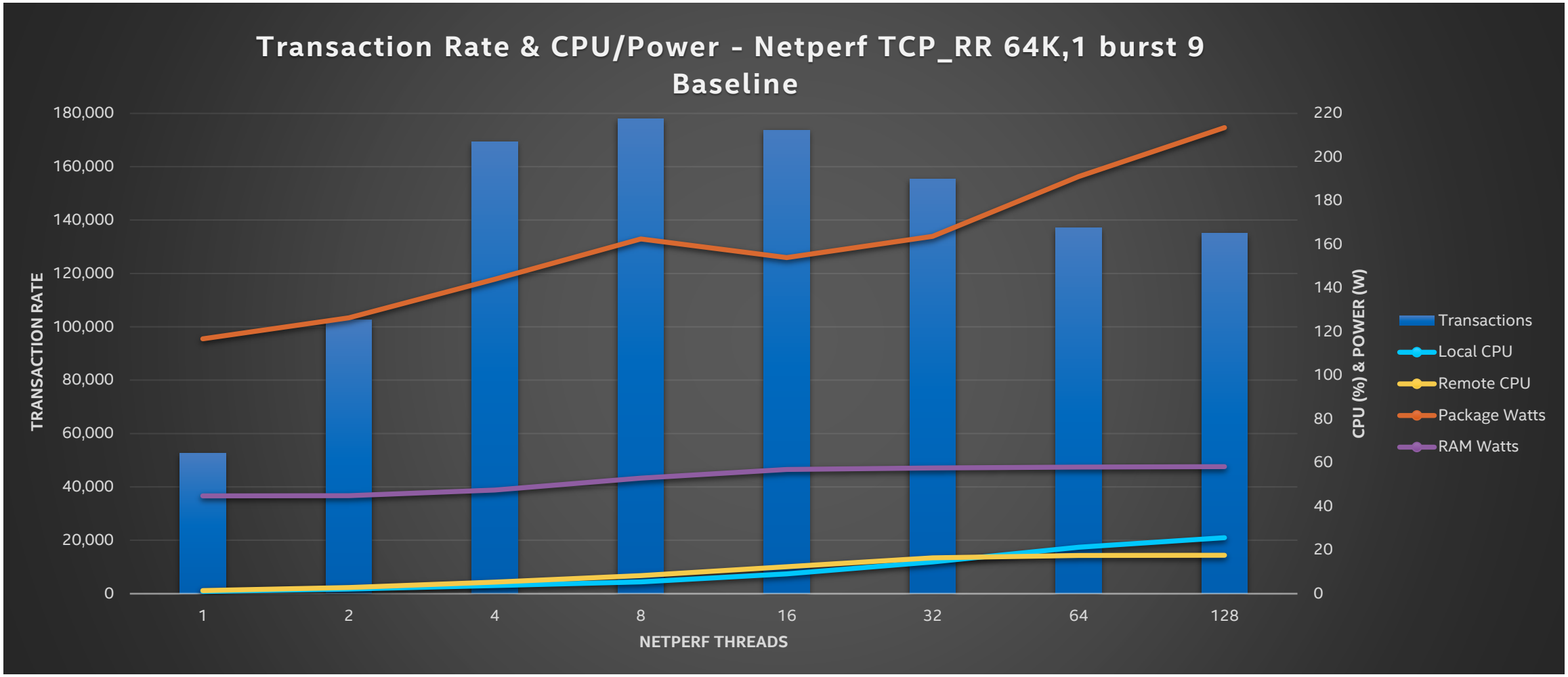
- Default configuration of queues is suboptimal
- Telling users to poll (C0) on all CPUs and queues to get performance is unacceptable
- PowerMAN PoC in a script yielded power savings in high thread count tests
- PowerMAN should be a part of the scheduler



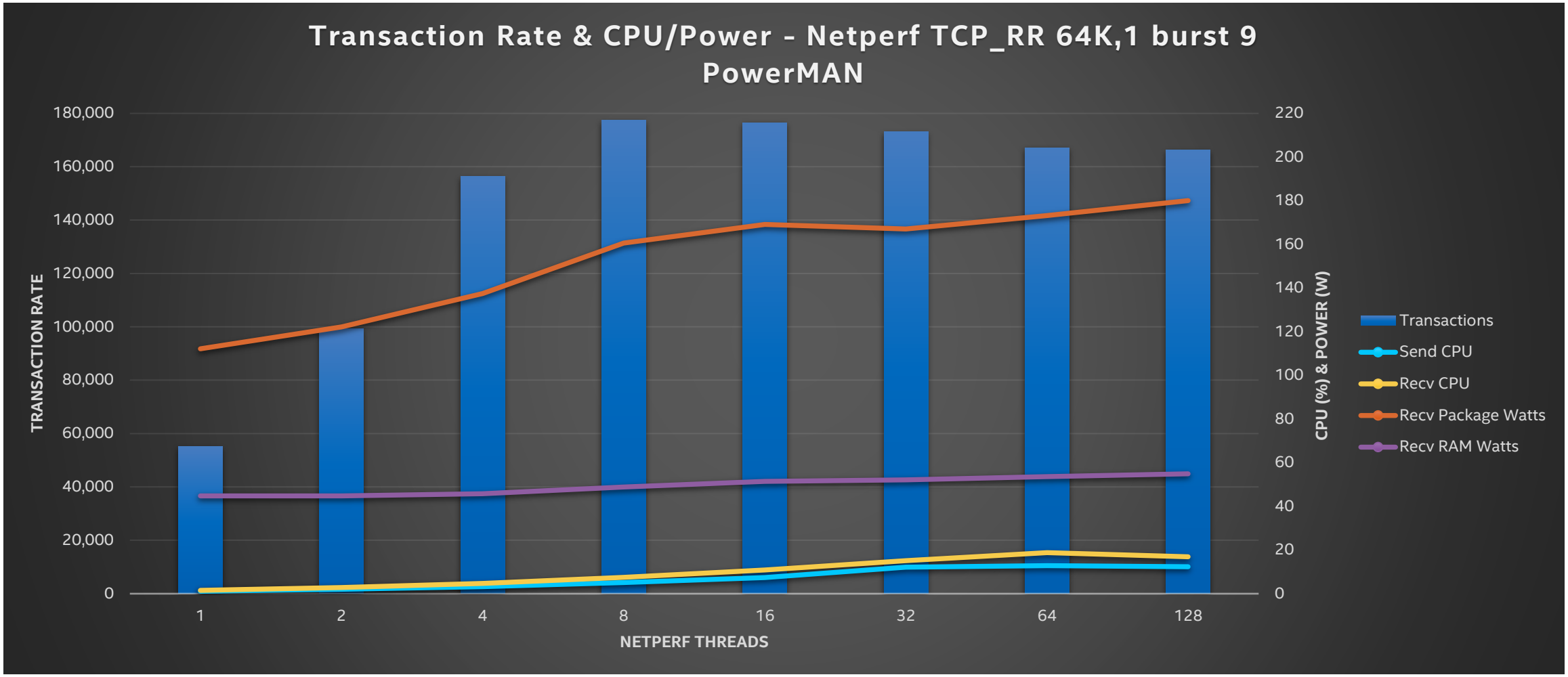

intel

BACKUP

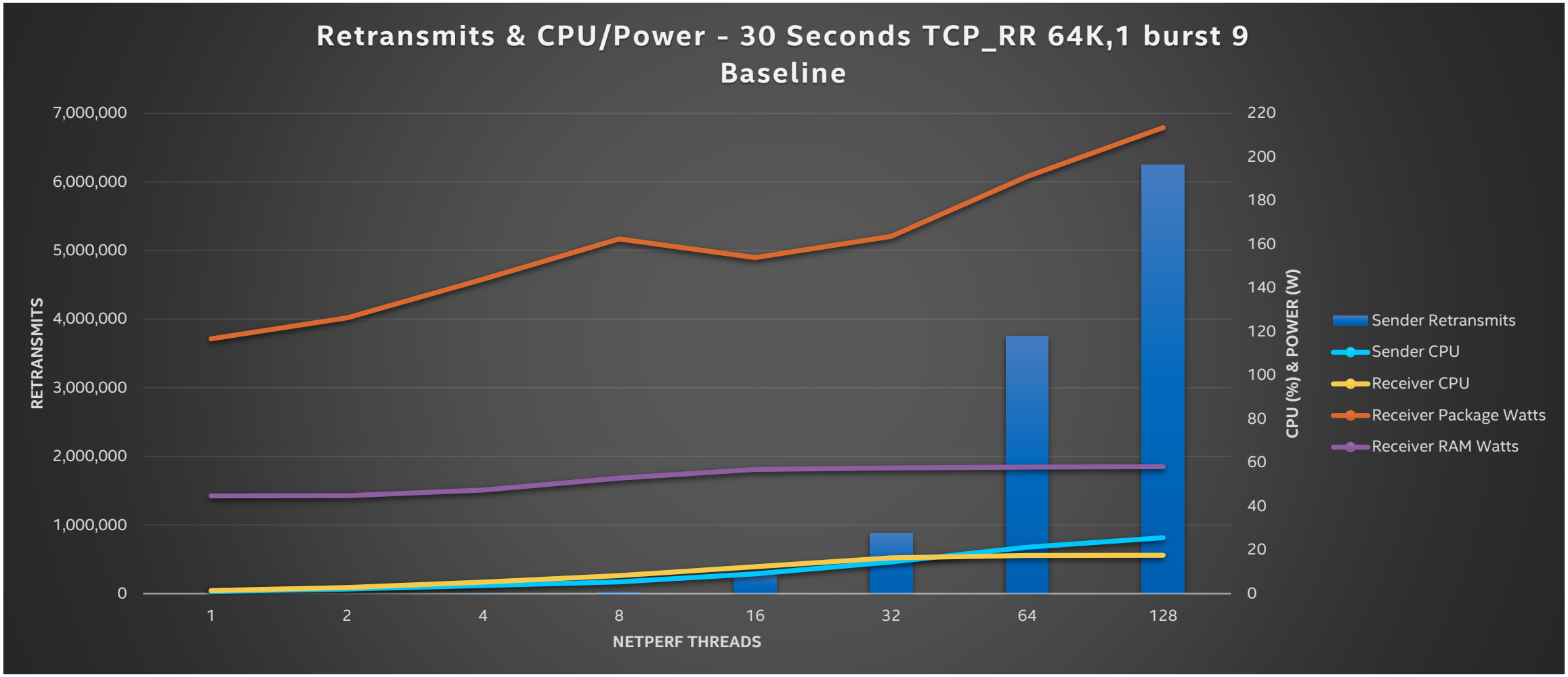
Baseline Receive - Transactions



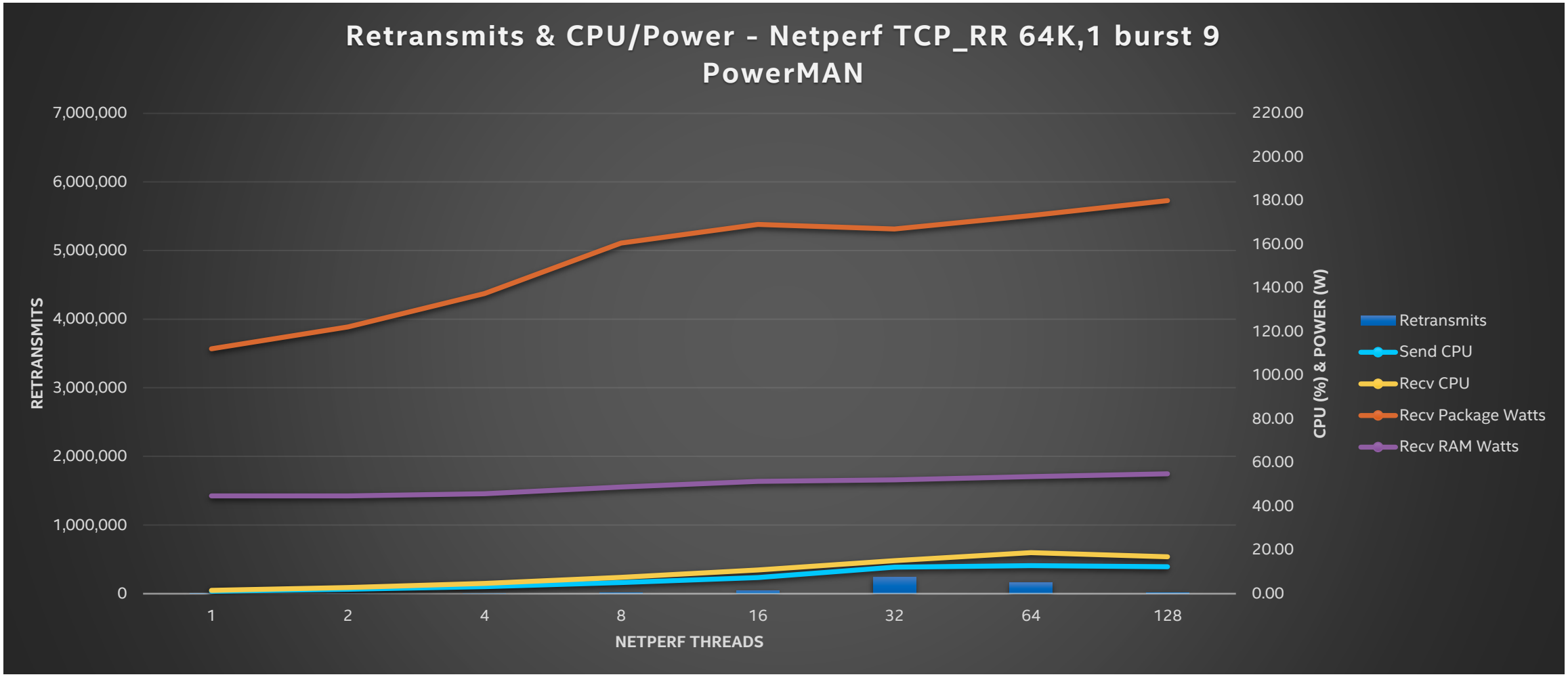
PowerMAN Receive – Transaction Rate



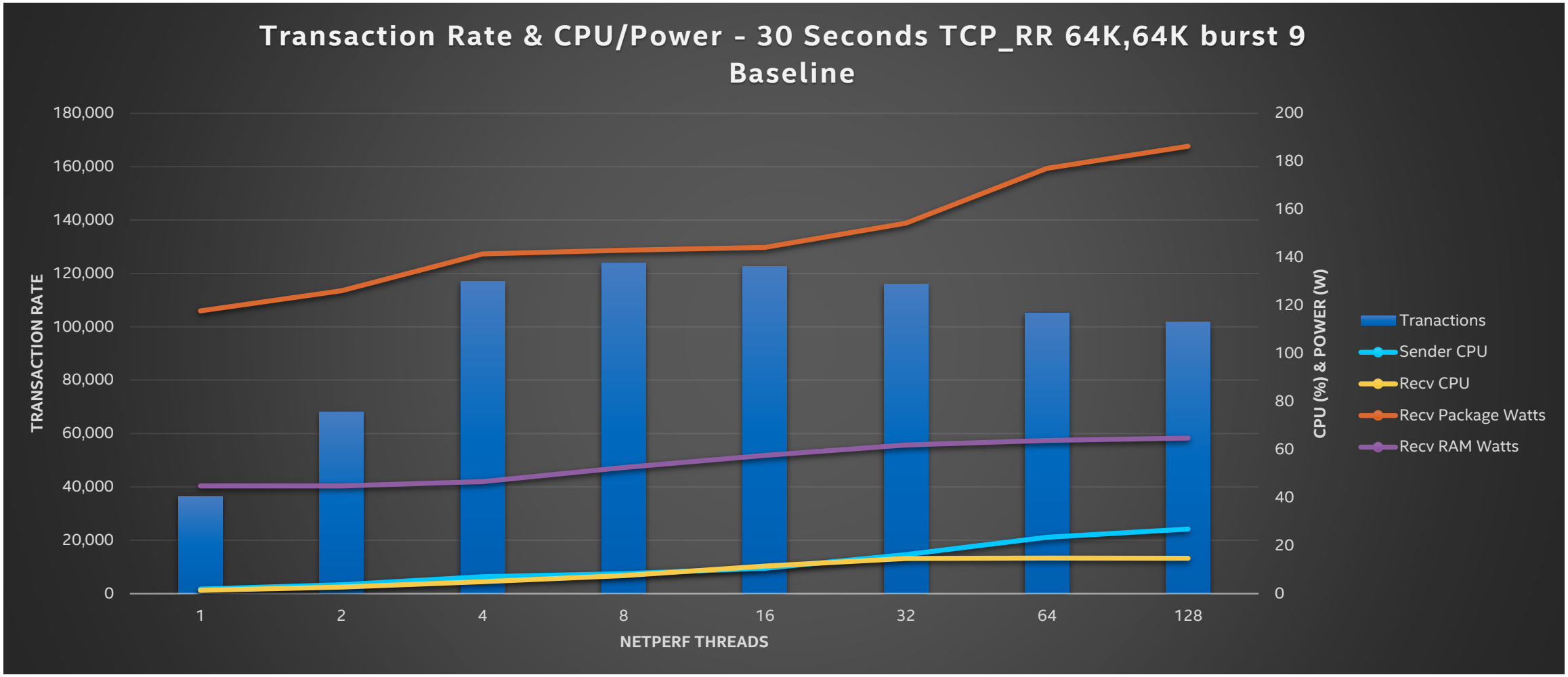
Baseline Receive - Drops



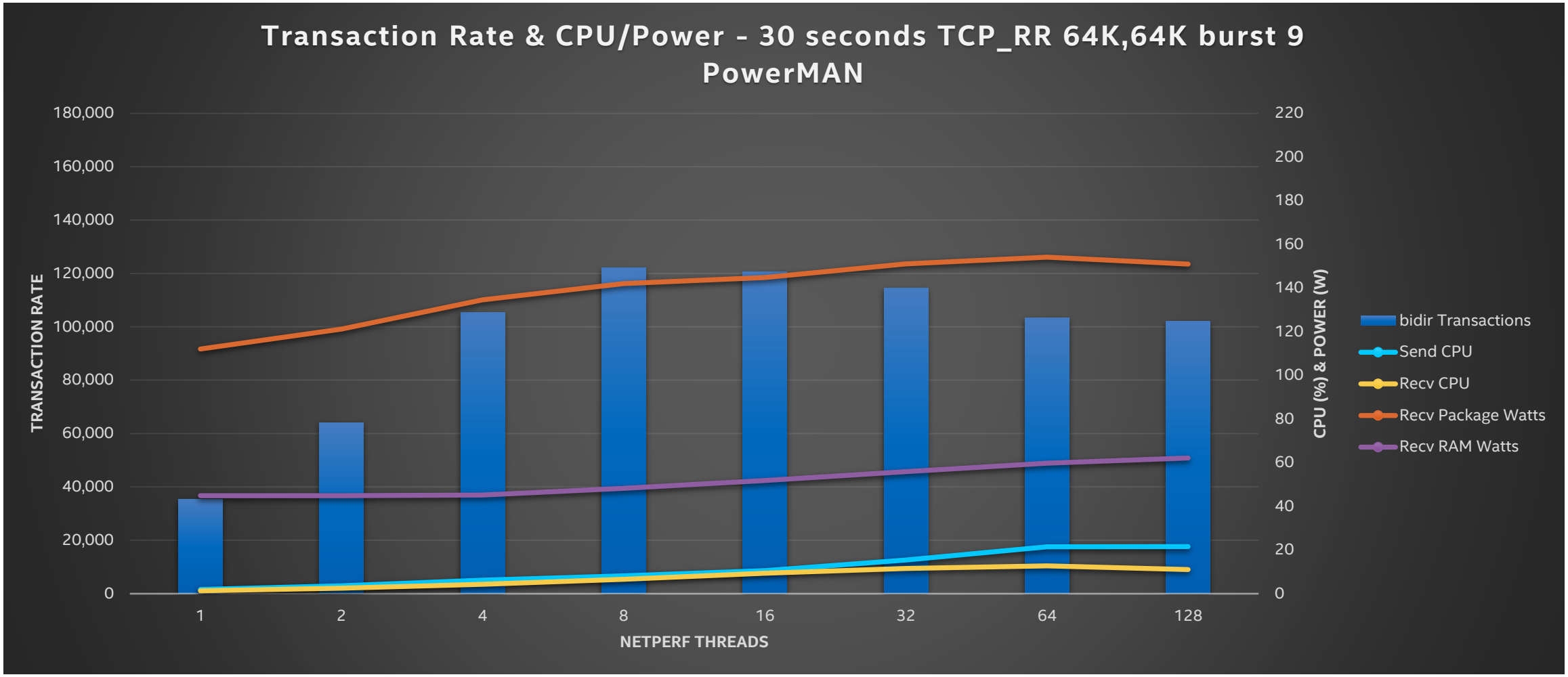
PowerMAN Receive - Drops



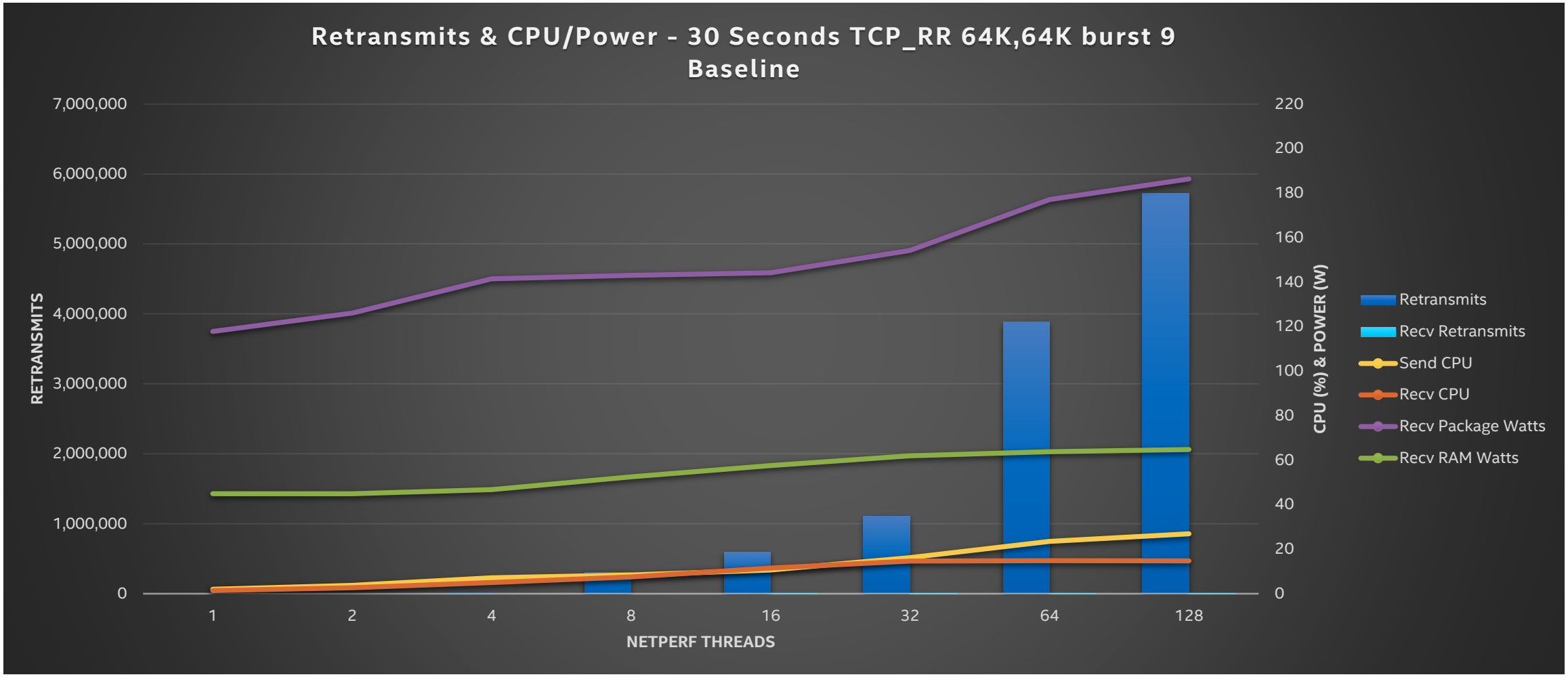
Baseline Bi-directional – Transaction Rate



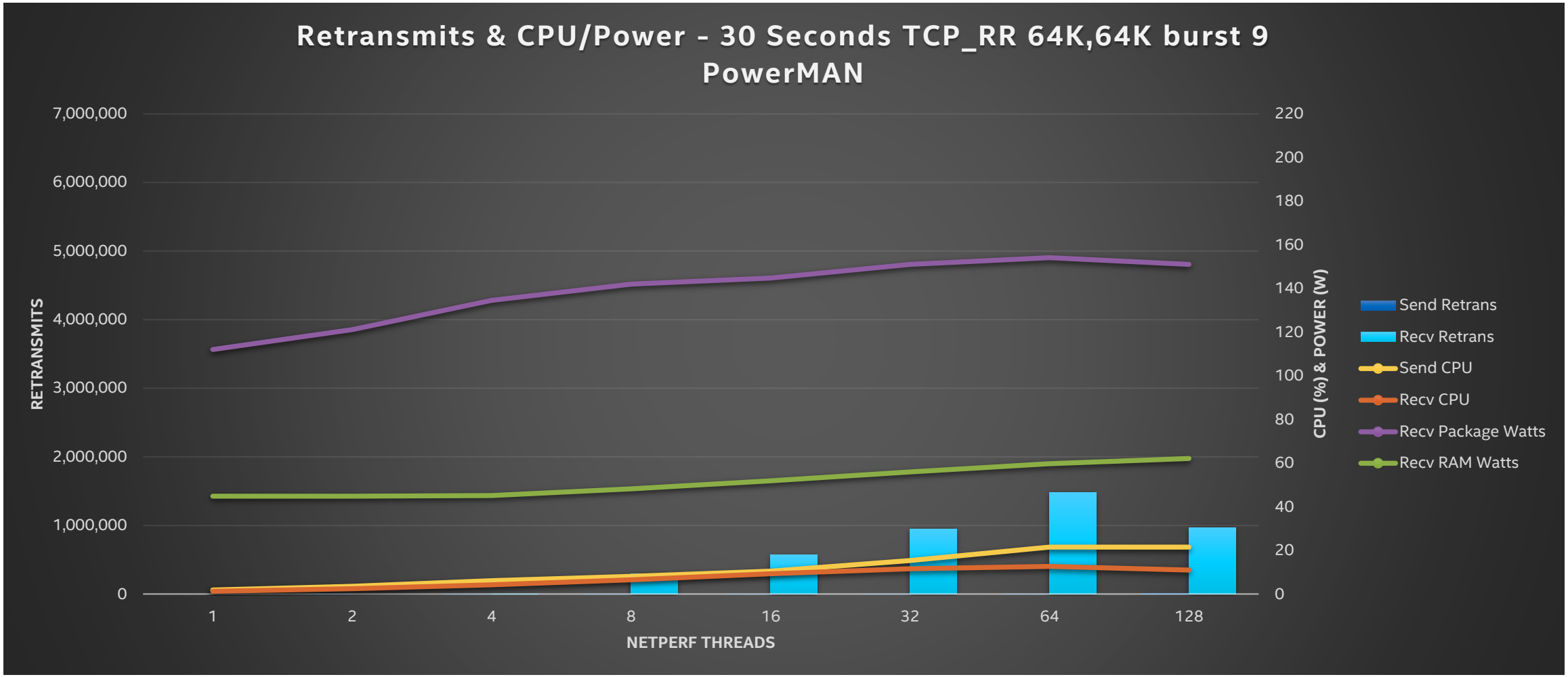
PowerMAN Bi-directional – Transaction Rate



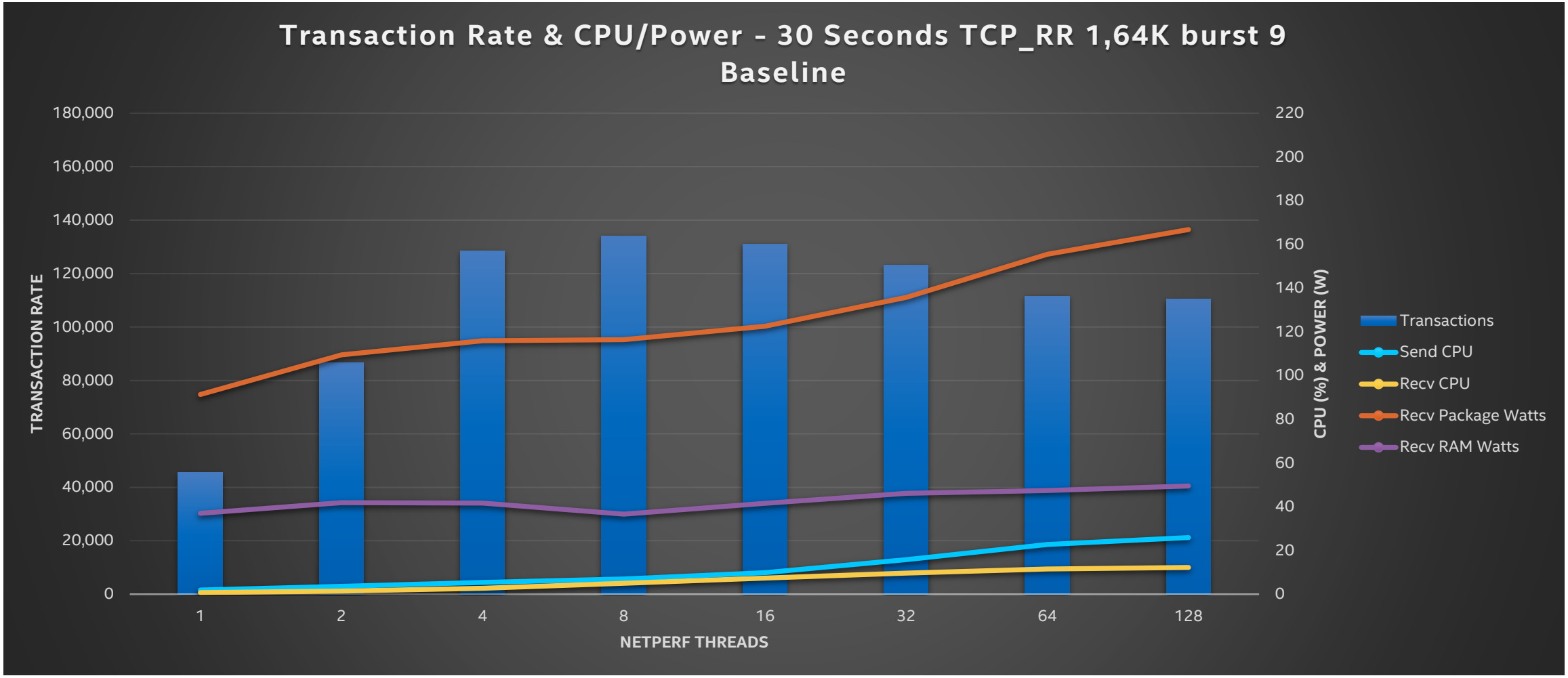
Baseline Bi-directional - Drops



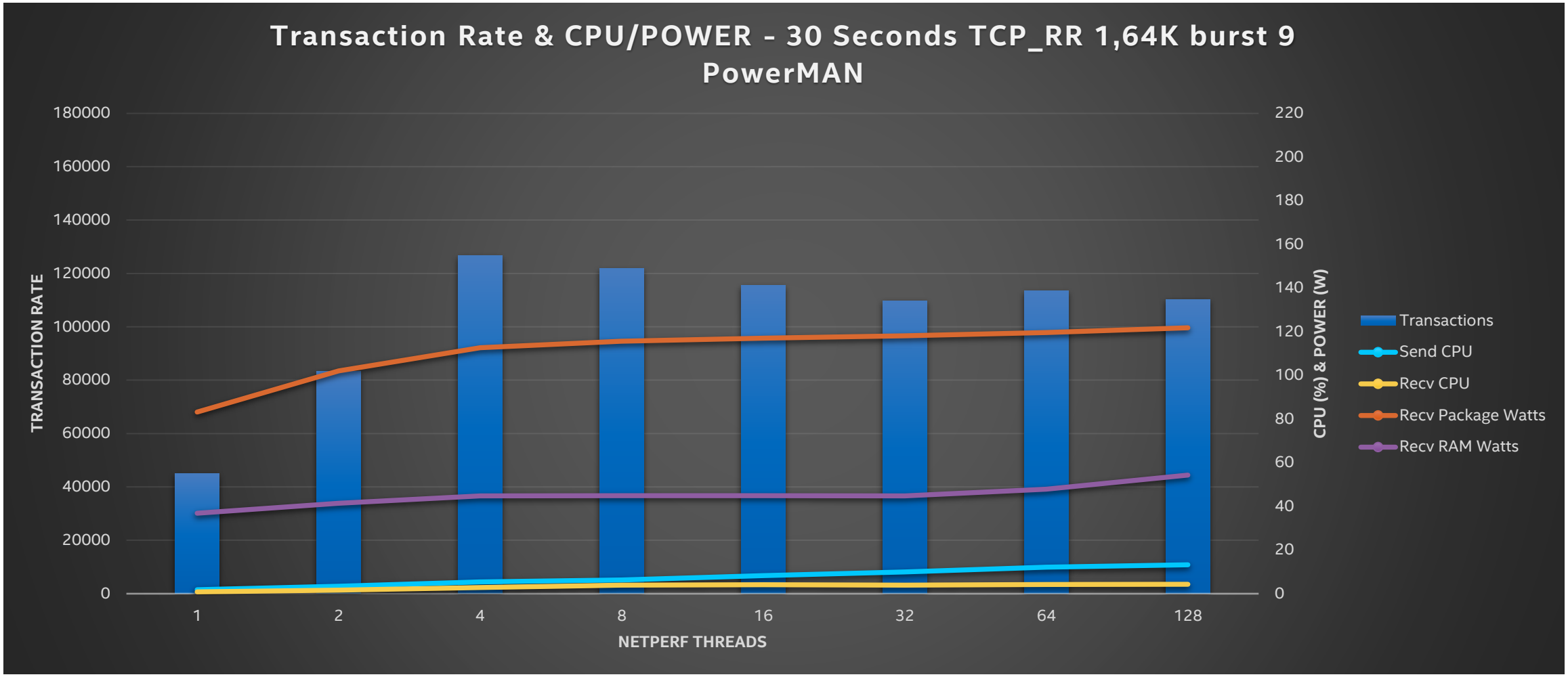
PowerMAN Bi-directional - Drops



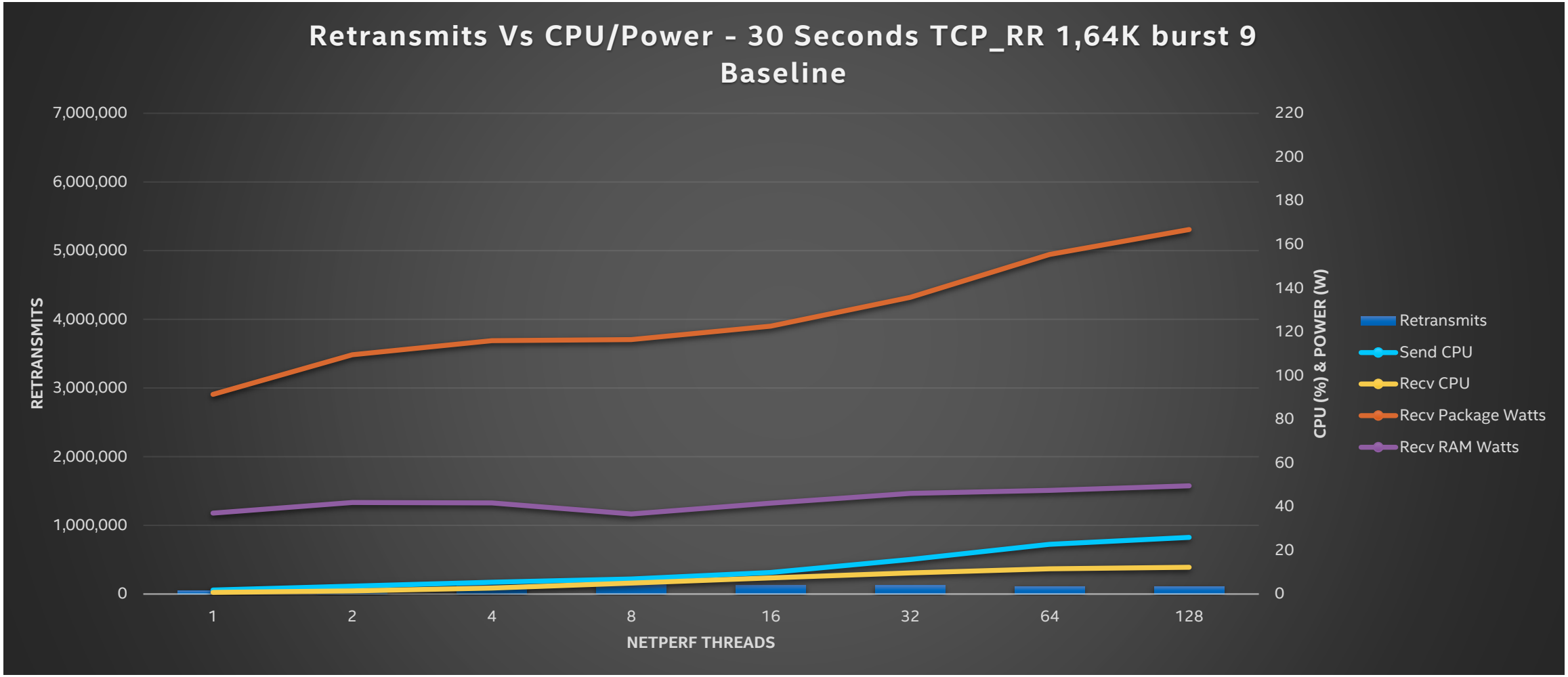
Baseline Transmit – Transaction Rate



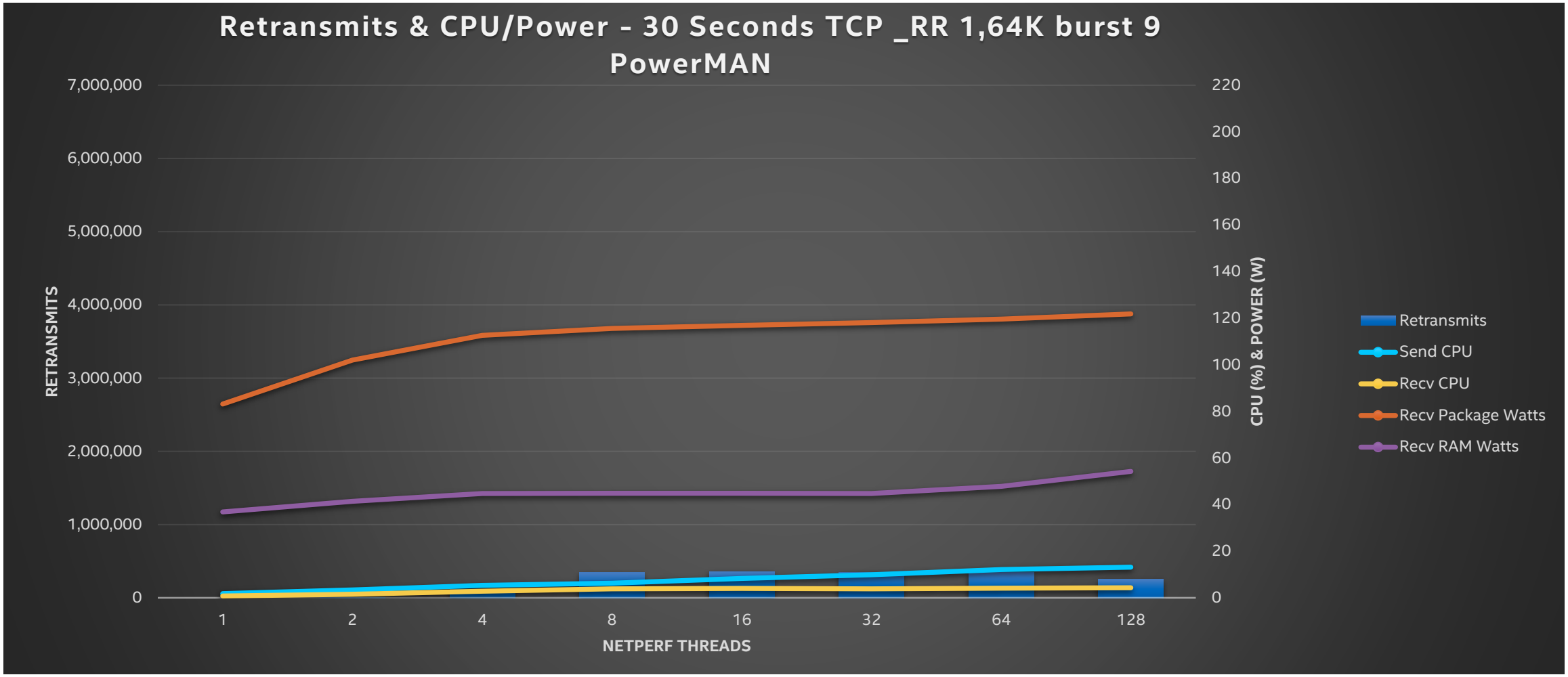
PowerMAN Transmit – Transaction Rate



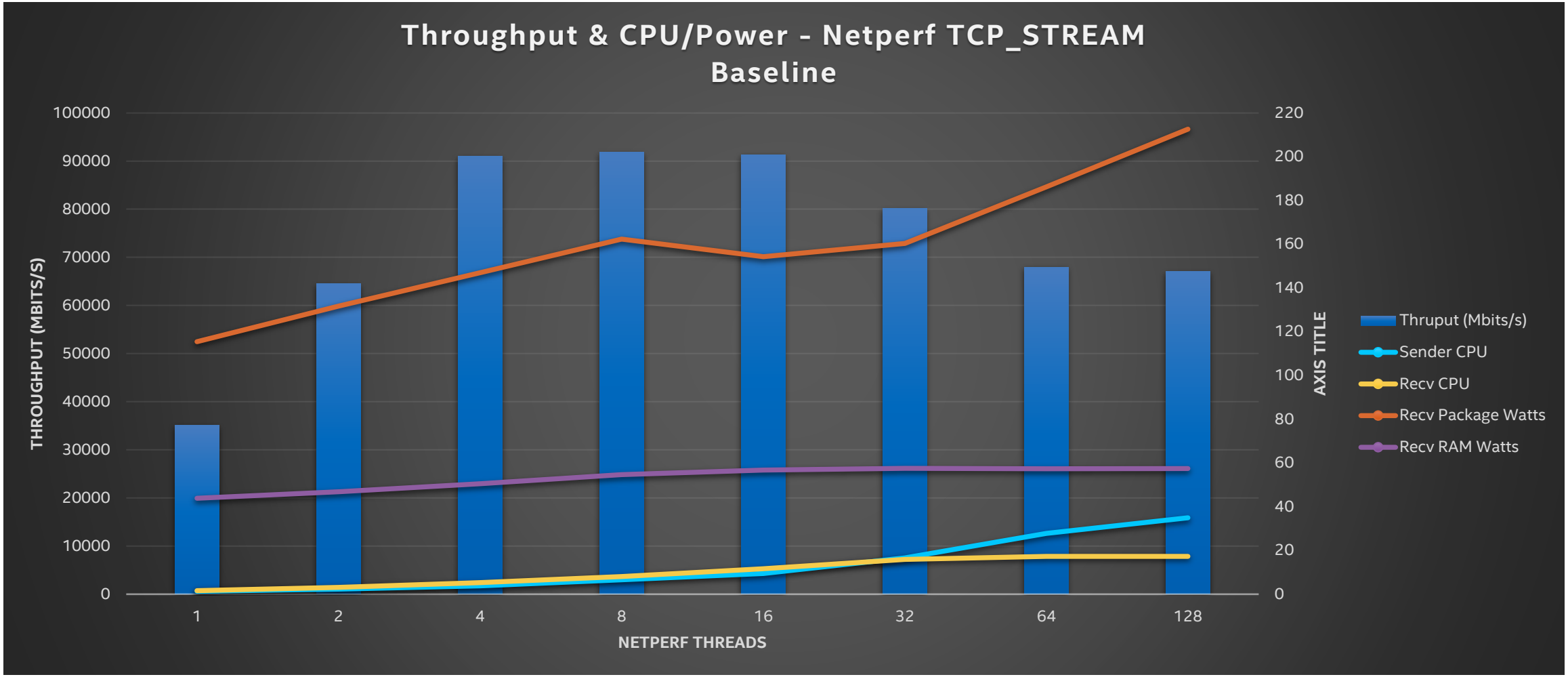
Baseline Transmit - Drops



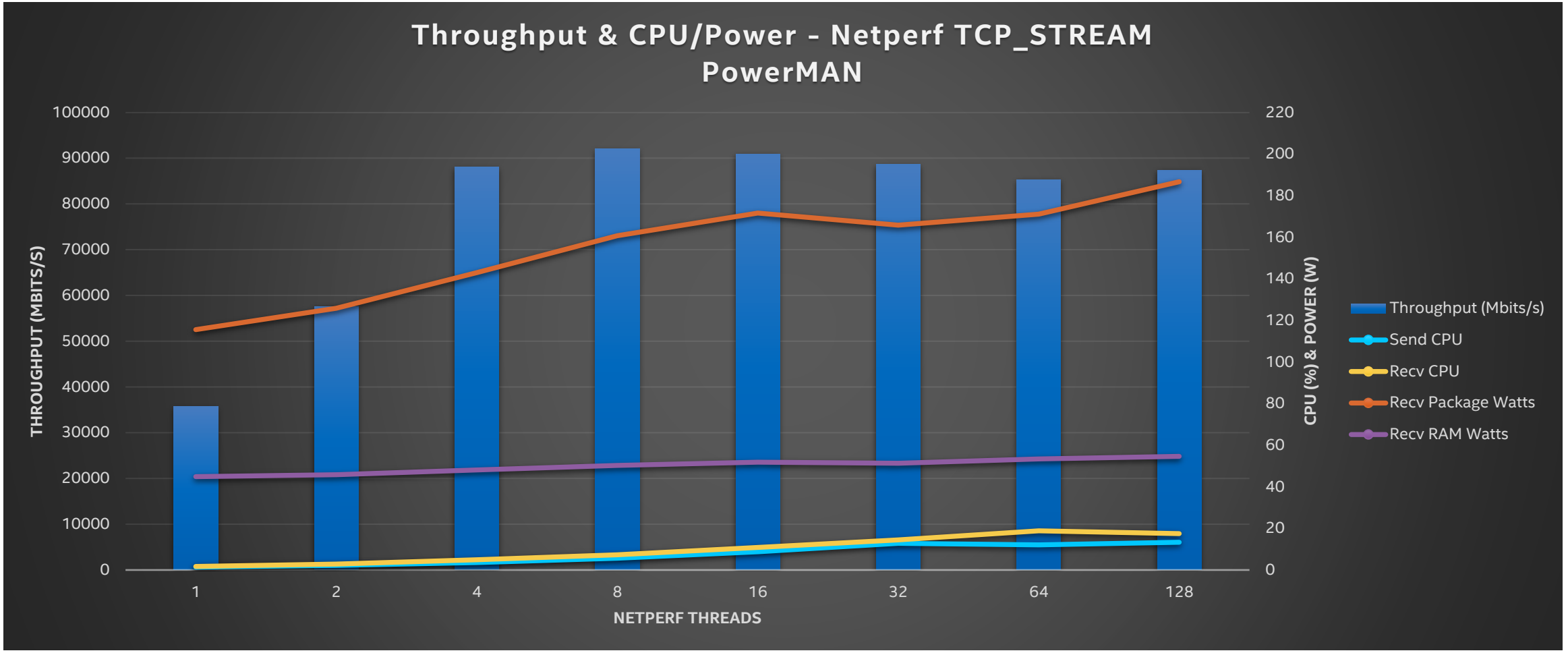
PowerMAN Transmit - Drops



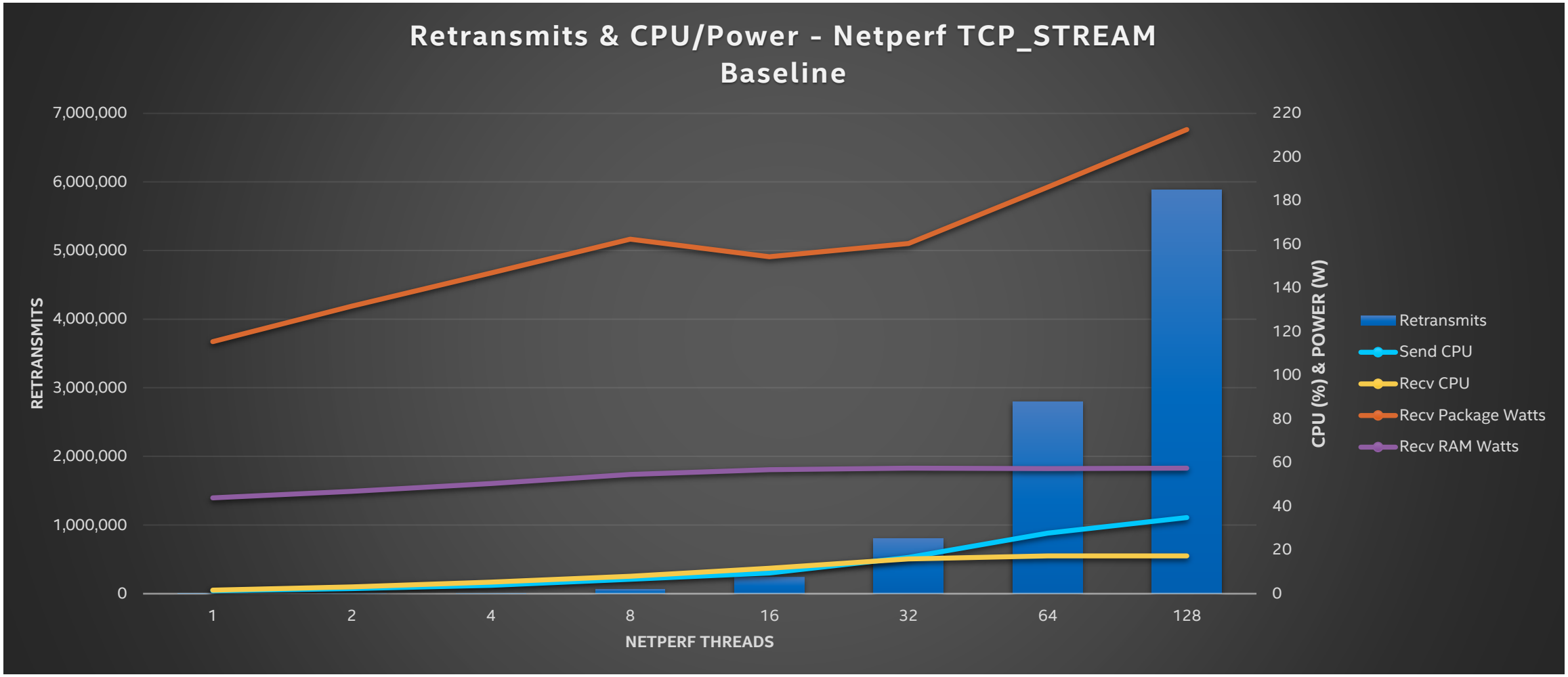
Baseline Stream Receive - Throughput



PowerMAN Stream Receive - Throughput



Baseline Stream Receive - Drops



PowerMAN Stream Receive - Drops

